Reviewer Pack — Minimal Index of Automorphism Groups in Hyperbolic Geometry …

Entry 450249a53b35 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 09:27:08 UTC

Minimal Index of Automorphism Groups in Hyperbolic Geometry Bounds Resolution Proof Width

Entry ID: `450249a53b35` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 09:27:08 UTC

1. Conjecture

Field A (mathematical branch): Hyperbolic Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For every CNF φ with n variables, the minimal index of automorphism groups (mφ) of its associated hyperbolic surface is linearly correlated with its resolution proof width w(φ), such that mφ = Θ(w(φ)).

Rationale (proposer's reasoning):

> Hyperbolic geometry provides a rich structure for studying symmetries in graphs, which may offer new insights into the complexity of resolving satisfiability. Automorphism groups capture these symmetries, and their size could reflect the difficulty of finding a proof.

Taxonomy category: `HyperbolicGeometry_ResolutionProofComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `99bbc33a2840f076`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between the minimal index of automorphism groups (mφ) and resolution proof width (w(φ)) across all CNFs is ≥ 0.7, with p-value ≤ 0.05. The conjecture is falsified if either the correlation coefficient < 0.5 or the p-value > 0.1.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"hyperbolic geometry" AND "resolution proof complexity" AND automorphism groups` - `"minimal index" OF automorphism groups IN hyperbolic geometry AND resolution proof width` - `automorphism groups in hyperbolic surfaces AND linear correlation with resolution proof complexity`

Top relevant hits considered: - [http://arxiv.org/abs/math/0504212v3] On the residual finiteness of outer automorphisms of relatively hyperbolic groups - [http://arxiv.org/abs/1511.09051v3] On automorphism groups of affine surfaces - [http://arxiv.org/abs/2104.14760v4] Automorphism and outer automorphism groups of Right-angled Artin groups are not relatively hyperbolic

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for i in range(n):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def compute_hyperbolic_surface(cnf):
        # Placeholder function to simulate the computation of automorphism groups
        # This is a dummy implementation and should be replaced with actual logic
        return len(cnf)  # Simplified for demonstration purposes

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        cnf = generate_cnf(n)
        m_phi = compute_hyperbolic_surface(cnf)
        w_phi = len(cnf) * n  # Simplified resolution proof width
        results.append((m_phi, w_phi))

    if not results:
        return {
```

```
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "empty_results"
        }

    m_phi_values = [m for m, _ in results]
    w_phi_values = [w for _, w in results]

    mean_m_phi = sum(m_phi_values) / len(m_phi_values)
    mean_w_phi = sum(w_phi_values) / len(w_phi_values)

    cov = sum((m - mean_m_phi) * (w - mean_w_phi) for m, w in results) / len(results)
    var_m_phi = sum((m - mean_m_phi) ** 2 for m in m_phi_values) / len(m_phi_values)
    var_w_phi = sum((w - mean_w_phi) ** 2 for w in w_phi_values) / len(w_phi_values)

    correlation_coefficient = cov / math.sqrt(var_m_phi * var_w_phi)

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(len(cnf) for _, cnf in results),
        "conjecture_holds": correlation_coefficient >= 0.7 and correlation_coefficient <= 1.3,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not all(result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_out_of_bounds\" first_f
    else:
        mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
        std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for res
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(result
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractior
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c6d813d0.py", line 76, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c6d813d0.py", line 66, in run_trial
    "n_max": max(len(cnf) for _, cnf in results),
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c6d813d0.py", line 66, in <genexpr>
    "n_max": max(len(cnf) for _, cnf in results),
                 ^^^^^^^^
TypeError: object of type 'int' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that the Pearson correlation coefficient and p-value could not be calculated to verify the conjecture. | next: Investigate the error in the test code and run it again to obtain the necessary data for calculating the Pearson correlation coefficient and p-value.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13146
2	preregistration	ollama_remote	glm4:latest	0	0	9326
3	novelty	ollama_remote	glm4:latest	0	0	8478
4	novelty	ollama_remote	glm4:latest	0	0	9627
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20365
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12719
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11107
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10641
9	judge	ollama_remote	glm4:latest	0	0	11659

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 107068 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/450249a53b35.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/450249a53b35.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/450249a53b35.tar.gz` (if generated)